My Current Rough Understanding of a Transformer Language Model

This is my current very rough understanding about the "transformer" thing. I have yet to understand many other details. For now, I am focusing on the high-level structure of this giant monster.

The note below is meant to record the basic architecture first. I will refine it later after I learn more.

High-level architecture of a Transformer language model

1. An input text is **tokenized** into a sequence of tokens:

$$t_1,\ t_2,\ \ldots,\ t_n.$$

For now, a token can be thought of roughly as a word.

2. There is a learnable embedding matrix:

$$E \in \mathbb{R}^{N_{\text{vocab}} \times D}.$$

Each token corresponds to one row of E. Therefore, every possible token is associated with a vector in

$$\mathbb{R}^D.$$

Initially, the entries of E are usually small random numbers. They become useful through training.

3. Each input token t_i is mapped to a row vector:

$$x_i^T \in \mathbb{R}^{1 \times D}.$$

Putting these row vectors together gives

$$X^{(0)} \in \mathbb{R}^{n \times D}.$$

Positional information is also added so that the model knows the order of the tokens.

4. From the current matrix $X^{(0)}$, the model creates three matrices:

$$Q \qquad \text{(query)},$$

$$K \qquad \text{(key)},$$

$$V \qquad \text{(value)}.$$

The query–key–value language comes from earlier attention used with RNN encoder–decoder models.[1] These are combined to form a score matrix S. The score matrix is then converted into an attention matrix A.

The attention matrix describes how much each token should use information from the other permitted tokens.

Using A and V, the model produces a new matrix Y. Each row of Y still corresponds to one token, but now incorporates information from other tokens in the text.

5. Attention is combined with a few additional neural-network operations to produce

$$X^{(1)}.$$

The matrix $X^{(1)}$ has the same overall size as $X^{(0)}$, but its token representations contain more contextual information.

6. This block of operations is repeated several times:

$$X^{(0)} \longrightarrow X^{(1)} \longrightarrow X^{(2)}$$

$$\longrightarrow \cdots \longrightarrow X^{(L)}.$$

The final matrix $X^{(L)}$ contains highly contextualized representations of all the input tokens.

7. To predict the next token, the model takes the last row of $X^{(L)}$ and applies a learnable affine transformation.

This produces one score for every possible token in the vocabulary. The scores are converted into probabilities, and the model chooses or samples the next token.

[1] In that older setting, an encoder represented an input sequence, and a decoder generated an output sequence one token at a time while using weighted combinations of the encoder representations. I will treat that historical machinery as a black box here. The words "query," "key," and "value" name computational roles; they are not literal database objects.

The whole architecture

In a very compressed form, the whole architecture is:

input text

$\downarrow$

sequence of tokens

$\downarrow$

embedding vectors

$\downarrow$

repeated Transformer blocks

$\downarrow$

scores for all possible next tokens

$\downarrow$

next token.

A short reminder to myself

At this stage, the most important picture is:

$$\text{tokens} \longrightarrow \text{vectors} \longrightarrow \text{contextualized vectors} \longrightarrow \text{next-token probabilities}.$$

The repeated Transformer blocks are where the representations become more and more contextual. I do not need to understand every internal detail yet in order to keep this high-level structure in mind.